Code Explanation

Workflow Runner for DPAddressChangeYUYU Code Explanation

The provided code is a Python module that serves as the entry point for running the DPAddressChangeYUYU workflow. It handles configuration, logging, and execution of the workflow using Apache Spark.

Overview

This code is responsible for setting up logging, retrieving configuration, creating a SparkSession, and running the DPAddressChangeYUYU workflow. It also includes error handling to ensure that any exceptions during execution are properly logged.

Step 1: Setting Up Logging

The setup_logging function is responsible for configuring the logging module. It creates a log directory if it does not exist and sets up a log file with a timestamp.

```
def setup_logging():
    log_dir = "/dbfs/mnt/logs/dp_address_change_yuyu"
    os.makedirs(log_dir, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    log_file = f"{log_dir}/dp_address_change_yuyu_{timestamp}.log"

    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s
',
        handlers=[
            logging.FileHandler(log_file),
            logging.StreamHandler(sys.stdout)
        ]
    )

    return logging.getLogger("DPAddressChangeYUYU")
```

Step 2: Retrieving Configuration

The get_config function retrieves configuration values from environment variables or uses default values if they are not set.

```
def get_config():
    return {
        "target_dir": os.environ.get("TARGET_FILE_DIR", "/dbfs/mnt/t
arget/files"),
        "batch_size": int(os.environ.get("BATCH_SIZE", "10000"))
    }
```

Step 3: Creating a SparkSession

In the main function, a SparkSession is created with specific configurations to enable Hive support and optimize Spark SQL performance.

```
spark = SparkSession.builder
    .appName("DPAddressChangeYUYU")
    .enableHiveSupport()
    .config("spark.sql.files.maxPartitionBytes", "134217728")
    .config("spark.sql.adaptive.enabled", "true")
    .config("spark.sql.shuffle.partitions", "200")
    .getOrCreate()
```

Step 4: Running the Workflow

The run_workflow function from the dp_address_change_yuyu module is called with the created SparkSession and retrieved configuration.

```
config = get_config()
logger.info(f"Using configuration: {config}")
run_workflow(spark, config)
```

Step 5: Error Handling and Exit

The code includes a try-except block to catch any exceptions during workflow execution. If an exception occurs, the error is logged, and the program exits with a non-zero status code.

```
try:
    # Workflow execution code here
except Exception as e:
    logger.error(f"Workflow failed: {str(e)}", exc_info=True)
    return 1
```

Step 6: Main Entry Point

The main function serves as the entry point for the script. It sets up logging, runs the workflow, and handles exceptions.

```
if __name__ == "__main__":
    sys.exit(main())
```